Certified Ethical Hacker (CEH)

End-of-Course Project

Project 4

SSH Brute-Force Detection

Log-Based Detection with Active iptables Response

Author	Anwar
Role	CEH Student
Platform	Kali Linux (VMware Workstation)
Date	July 2, 2026
Status	Complete

Table of Contents

1. Objective

The goal of this project is to build a working detection capability for SSH brute-force attacks. A brute-force attack repeatedly attempts to guess account credentials, leaving a distinctive trail of failed authentication events in the system log. The objective is to generate that trail under controlled lab conditions, then develop a Python tool that reads the authentication log, recognises the attack pattern, and raises an alert identifying the offending source.

The project covers the full defensive workflow end to end: standing up a target service, launching a real (but contained) brute-force attack, verifying the resulting log evidence, writing a parser and detection engine, confirming the tool successfully flags the attack, and finally extending it from passive detection into active response — automatically blocking the offending source with a firewall rule. Particular attention is given to correctly parsing modern OpenSSH log formats, and to generating an attacker address that can be safely blocked without disrupting the host.

1.1 Scope and Ethical Note

All activity described in this report was performed inside an isolated VMware lab environment against a machine owned and operated by the author. The brute-force attack targeted a purpose-built test account on the local loopback interface (127.0.0.1). No external, third-party, or production system was involved at any point. The techniques are documented for defensive and educational purposes as part of the CEH curriculum.

2. Lab Environment

The project was completed entirely on the Kali Linux virtual machine, which served as both the attack host and the detection host. This self-contained design keeps the exercise reproducible and confined to a single system.

Hypervisor	VMware Workstation
Detection / Target Host	Kali Linux
Hostname	`kalibaba@Kalibaba`
Kali IP Address	`192.168.229.159`
Target Service	OpenSSH server (sshd), TCP port 22
Attack Target	`127.0.0.1 (loopback)`
Attack Tool	Hydra v9.7; sshpass + ssh -b (source-bound loop)
Simulated Attacker IP	`127.0.0.2 (loopback alias, safe to block)`
Response Mechanism	iptables (INPUT chain DROP rule)
Detection Language	Python 3
Log Source	`/var/log/auth.log`

A second virtual machine, Windows 10 (192.168.229.164), exists in the lab but was not required for this project; the detection scenario is fully self-contained on the Kali host.

3. Methodology

The following steps were carried out in sequence on the Kali Linux terminal. Each command is shown exactly as executed, followed by a screenshot of the result and an explanation of what the step accomplishes.

Step 1 — Verify the SSH Service State

Before an SSH brute-force attack can generate log evidence, the SSH server must be running to receive and reject the login attempts. The first command inspects the current state of the service.

```
sudo systemctl status ssh
```

The output reported the service as loaded but inactive (dead) and disabled, meaning OpenSSH was installed on the system but not currently running. This is the expected default state on Kali Linux and confirms the service simply needs to be started.

Figure 1. *Output of systemctl status ssh showing the service inactive (dead) before startup.*

Step 2 — Start the SSH Server

With the service confirmed as installed, it was started so that it would begin listening for inbound connections. The status was then re-checked to confirm the change.

```
sudo systemctl start ssh
sudo systemctl status ssh
```

The service state changed to active (running), and the log lines confirmed the daemon was now listening on 0.0.0.0 port 22 (all interfaces). The SSH server is now ready to receive the brute-force attempts.

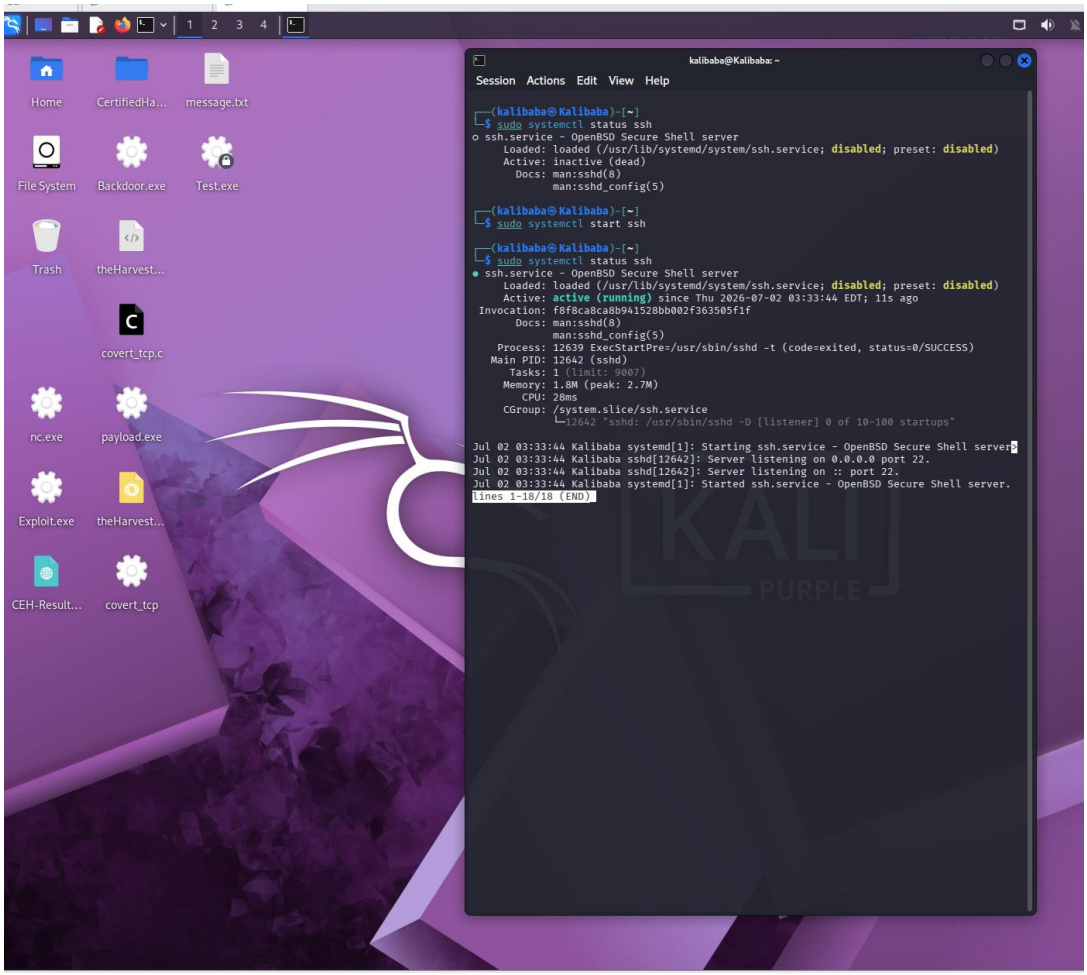

Figure 2. *SSH service checked (inactive), started, and re-verified as active (running), listening on port 22.*

Step 3 — Create a Dedicated Test Account

To avoid attacking a real user account, a disposable test account named victim was created with a known weak password. This account exists purely to be the target of the controlled attack.

```
sudo useradd -m victim
sudo passwd victim
```

The useradd -m command created the account together with a home directory. The passwd command then set a deliberately weak password (password123) for it. The system confirmed the change with the message "password updated successfully". Using a known weak password ensures the attack produces a realistic sequence of failed attempts followed by one success.

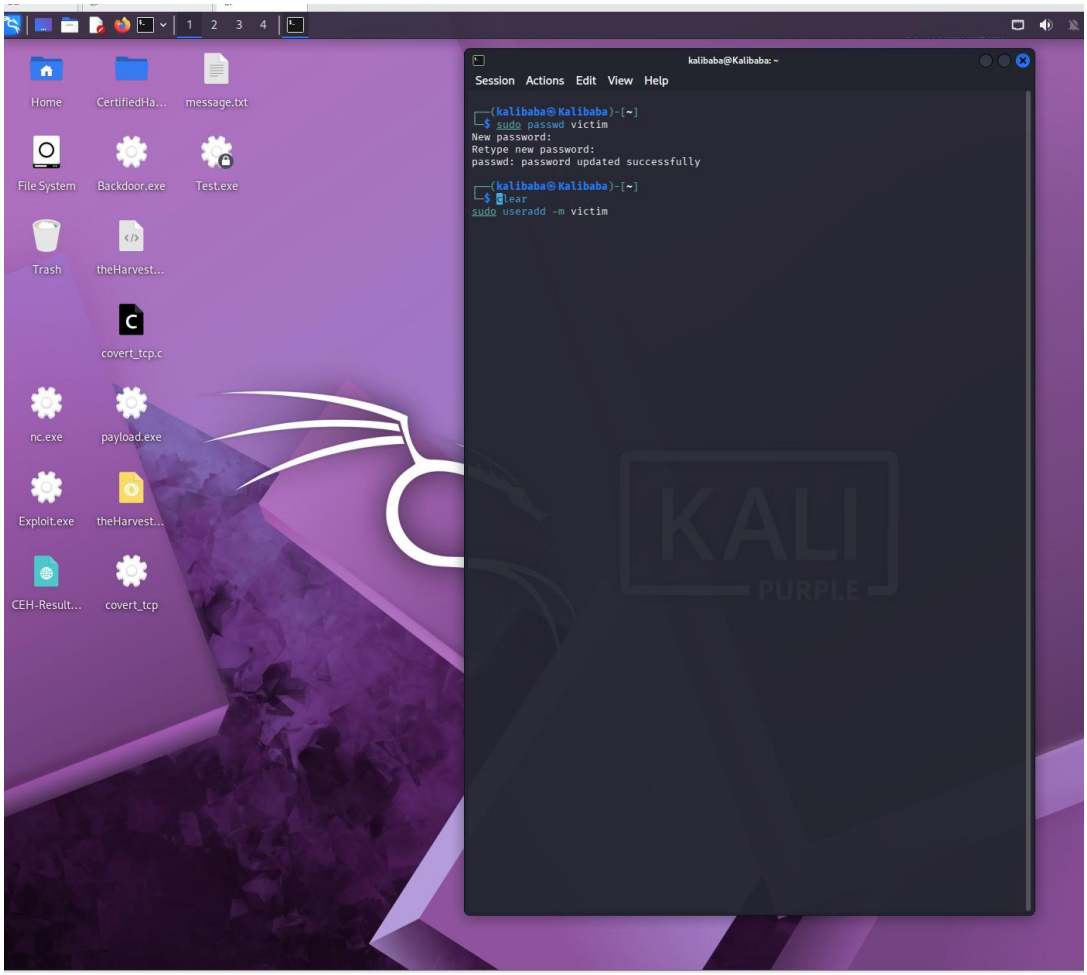

Figure 3. *Creation of the victim account and successful password assignment ("password updated successfully").*

Step 4 — Build the Attack Wordlist

Hydra requires a list of candidate passwords to try. A small wordlist of eight entries was created, containing seven common wrong guesses followed by the account's real password in the final position. This produces a clean sequence of failed attempts ending in a single success.

```
printf '123456\nadmin\nletmein\nqwerty\nroot\ntoor\npassword\npassword123\n' >
~/wordlist.txt
cat ~/wordlist.txt
```

The cat command verified the file held eight passwords, one per line, with password123 in the last position. This ordering guarantees the attack logs seven failures before it succeeds — exactly the pattern the detector is designed to catch.

```
  kalibaba@Kalibaba: ~
$ cat ~/wordlist.txt
123456
admin
letmein
```

```
qwerty
root
toor
password
password123
```

Figure 4. *Contents of wordlist.txt confirming eight passwords with the correct one (password123) last.*

Step 5 — Launch the Brute-Force Attack with Hydra

Hydra was pointed at the local SSH service using the victim username and the wordlist. Each incorrect guess causes the SSH server to write a "Failed password" entry to the authentication log — the raw material the detector will analyse.

```
hydra -l victim -P ~/wordlist.txt ssh://127.0.0.1 -t 4
```

The command flags are as follows:

- -l victim — the single username to attack.
- -P ~/wordlist.txt — the password list to try in turn.
- ssh://127.0.0.1 — the target service and host (local loopback).
- -t 4 — run four parallel tasks, keeping the attempts ordered and the log clean.

Hydra worked through all eight login attempts and reported "1 valid password found", correctly recovering login: victim / password: password123. The seven failed attempts preceding the success were recorded in the authentication log.

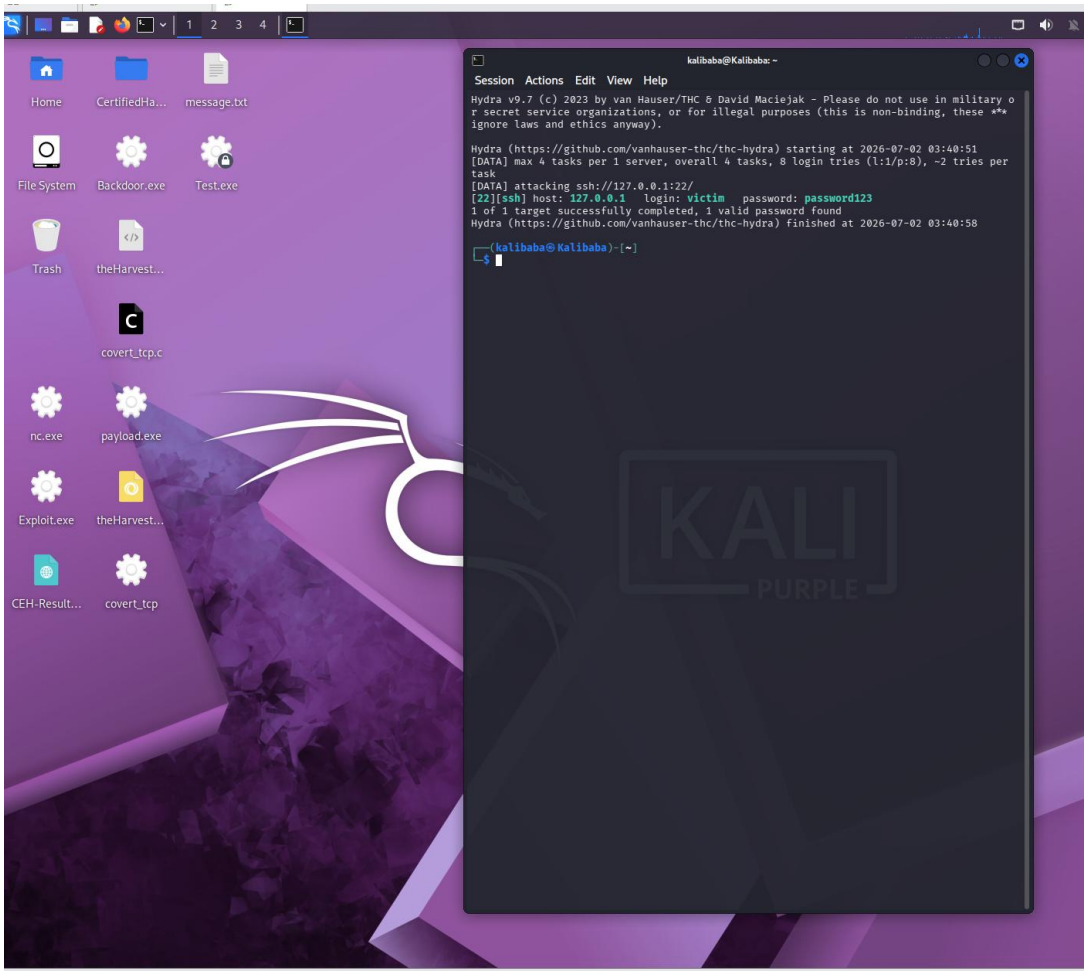

Figure 5. *Hydra completing the attack and recovering login: victim / password: password123 (1 valid password found).*

Step 6 — Verify the Failed Attempts in the Log

This step is critical: it inspects the exact format of the log lines the SSH server produced. The precise wording and structure of these lines dictate how the detector's parser must be written.

```
sudo grep "Failed password" /var/log/auth.log | tail -n 20
```

A representative failed-login line from the log reads:

```
2026-07-02T03:40:53.836573-04:00 Kalibaba sshd-session[16310]: \
  Failed password for victim from 127.0.0.1 port 49884 ssh2
```

Inspecting these live lines revealed three format characteristics of modern OpenSSH that any parser must account for. These are analysed in detail in Section 5, as they were the root cause of the earlier detection failure. A supplementary check using journalctl confirmed the same events from the systemd journal.

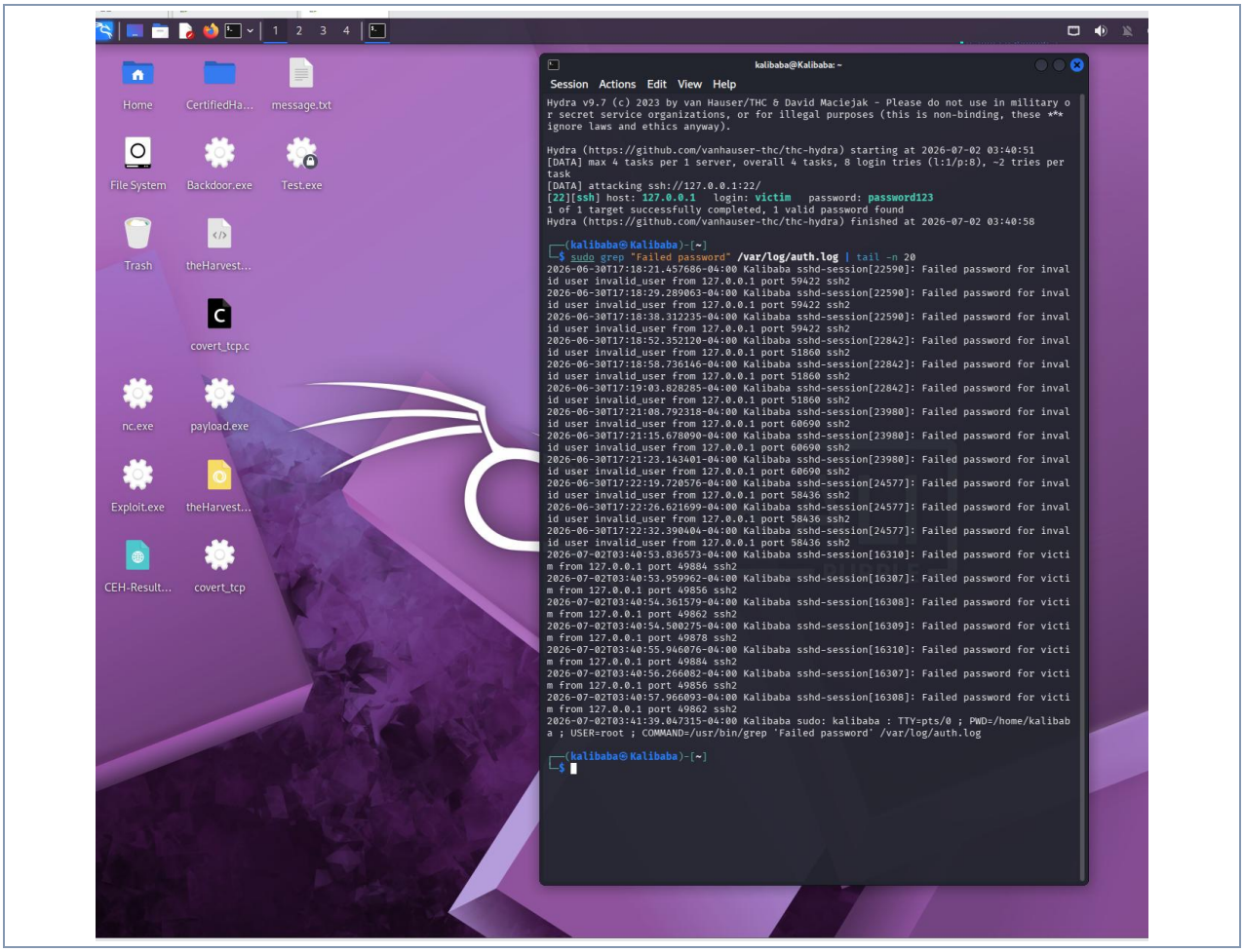

Figure 6. *Failed-password entries in /var/log/auth.log, showing the sshd-session process name and ISO 8601 timestamps.*

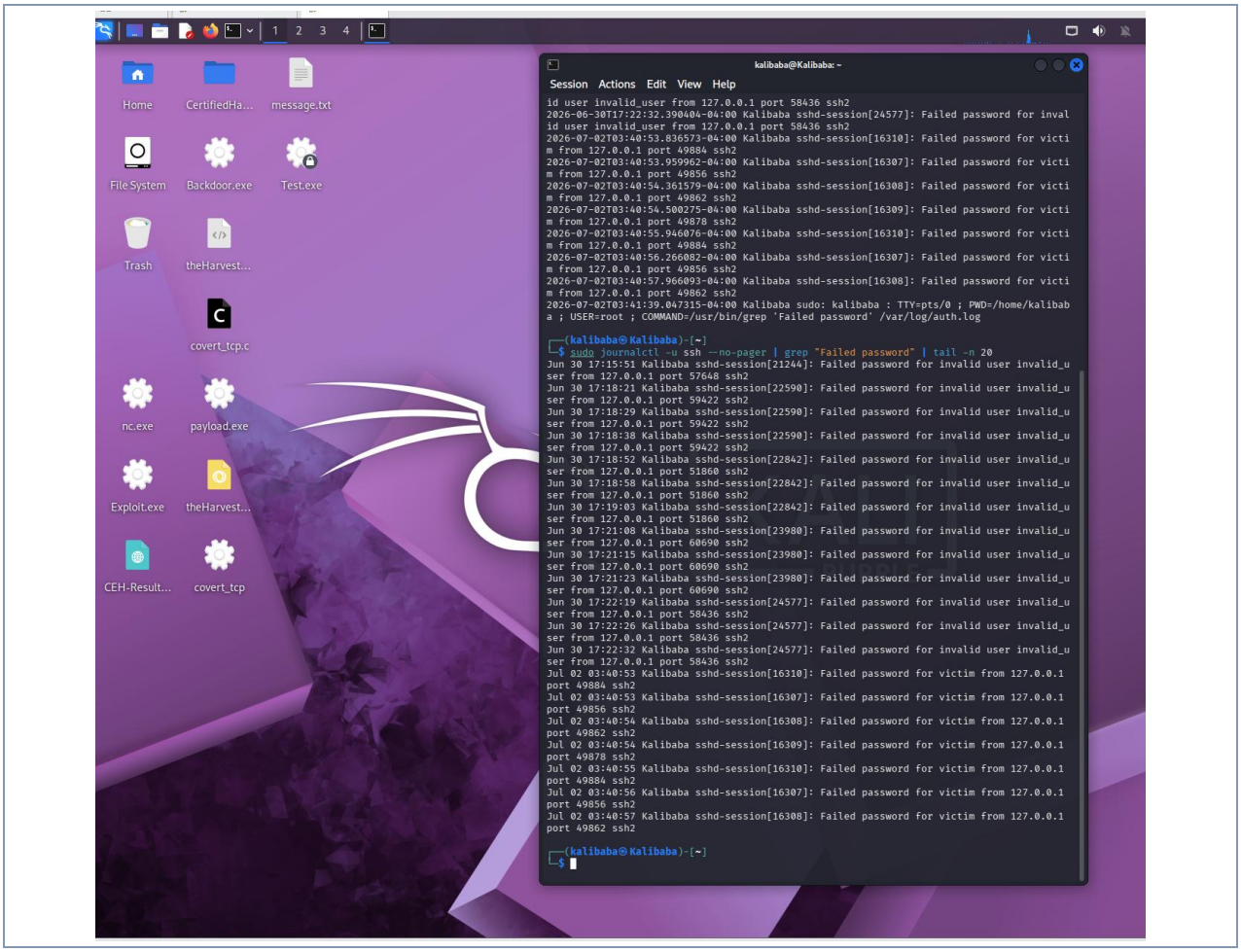

Figure 7. *Supplementary confirmation of the same failed-login events from the systemd journal (journalctl -u ssh).*

Step 7 — Create the Detector Script

A Python 3 script named detector.py was written to parse the authentication log, group failed attempts by source IP, and raise an alert when any single IP exceeds a configurable threshold of failures within a sliding time window. The script was created and edited using the nano text editor.

```
nano ~/detector.py
```

The complete source code is provided in Appendix B, and the detection logic — including the corrected regular expression — is explained in Section 4. The file was saved and its header verified with a cat command.

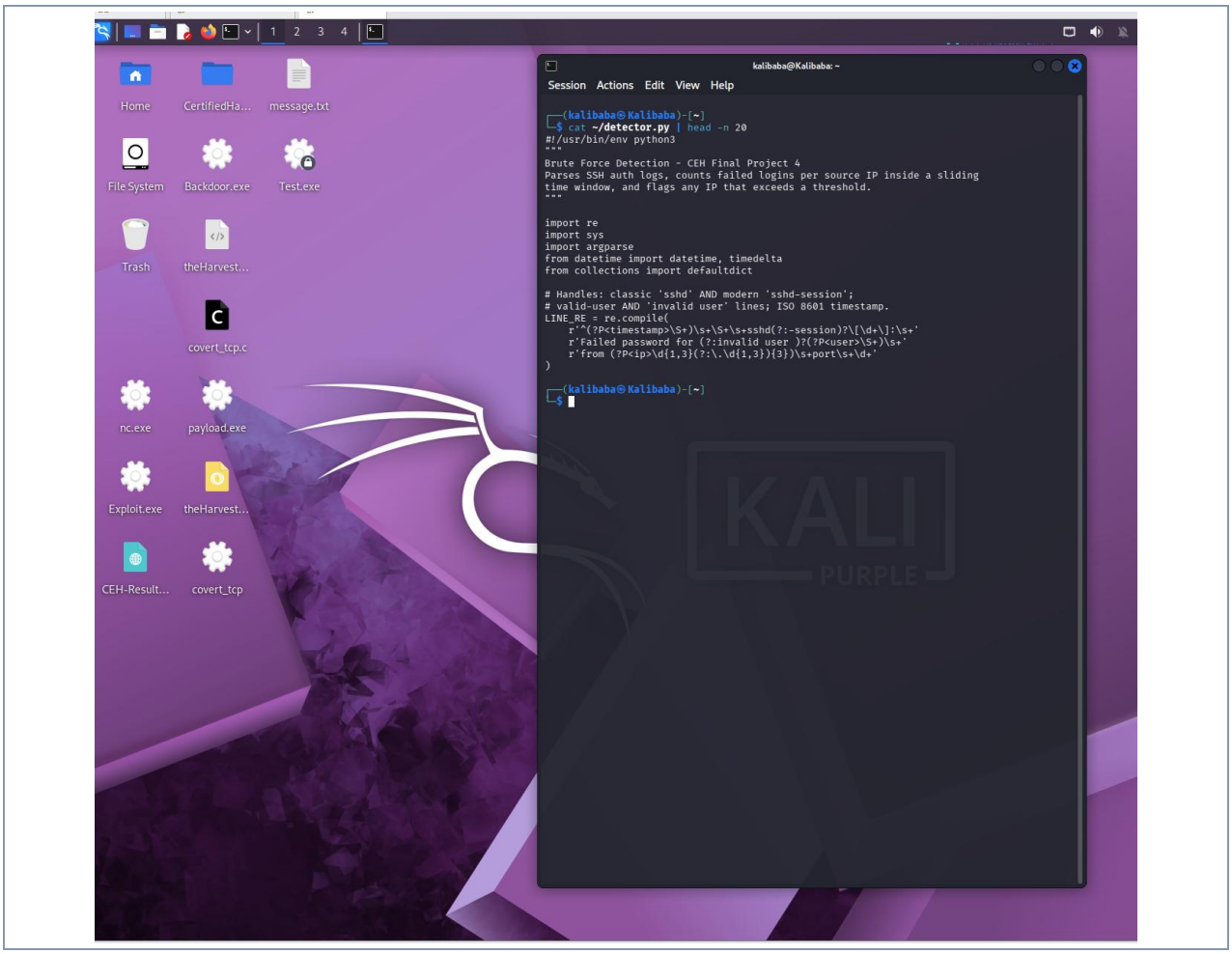

Figure 8. *The saved detector.py script, showing the header, imports, and the corrected regular expression.*

Step 8 — Run the Detector Against the Live Log

Finally, the detector was executed against the real authentication log with a detection rule of five or more failed attempts from a single IP within a 60-second window.

```
sudo python3 ~/detector.py -t 5 -w 60
```

The tool parsed 70 failed login attempts and raised a brute-force alert against 127.0.0.1, reporting a peak of 8 failures inside the 60-second window and correctly listing the targeted usernames. This confirms the detector works end to end. The full output and its interpretation appear in Section 6.

```
  kalibaba@Kalibaba: ~
$ sudo python3 ~/detector.py -t 5 -w 60
[*] Parsed 70 failed login attempt(s) from /var/log/auth.log
[*] Rule: >= 5 failures within 60s from one IP

[ALERT] Brute-force from 127.0.0.1
        8 failed attempts within 60s (total 70 from this IP)
        window: 2026-06-30 12:58:35.760400-04:00 ->
```

```
                2026-06-30 12:58:39.156306-04:00
        targeted user(s): invalid_user, kalibaba, victim
```

Figure 9. *The detector raising a brute-force ALERT against 127.0.0.1 with attempt count, time window, and targeted users.*

4. Detection Logic

The detector operates in three stages: parse, group, and evaluate.

4.1 Parsing

Each line of the authentication log is tested against a single regular expression that isolates three fields from every failed-login event: the timestamp, the targeted username, and the source IP address. Lines that do not match are ignored, so unrelated log noise is discarded automatically.

4.2 Grouping and Sliding Window

Matched events are grouped by source IP. For each IP, the timestamps are sorted and a sliding-window algorithm scans for the densest cluster of failures: it advances an end pointer through the events and moves a start pointer forward whenever the span between the two exceeds the configured window (default 60 seconds). The largest number of events falling inside any single window is recorded for that IP.

4.3 Alerting

If the densest window for an IP meets or exceeds the threshold (default 5), the IP is flagged. The alert reports the source IP, the peak count inside the window, the total failures seen from that IP, the exact time span of the densest burst, and the set of usernames that were targeted. This gives an analyst everything needed to triage the event at a glance.

5. Root-Cause Analysis: The Log-Parsing Fix

The central technical challenge of this project was that an initial version of the detector failed to match any events, despite the log clearly containing failed-login lines. Inspecting the live log in Step 6 revealed three characteristics of modern OpenSSH logging that a conventional, textbook SSH regex does not anticipate.

5.1 The Three Format Differences

#	Characteristic	Why a Naive Regex Fails
1	Process name is sshd-session, not sshd	OpenSSH 9.8+ split the daemon into a per-connection worker named sshd-session. A pattern anchored on 'sshd[' matches none of the lines.
2	ISO 8601 timestamp	Lines begin with 2026-07-02T03:40:53.836573-04:00, not the classic 'Jul 2 03:40:53' syslog format. A parser expecting the old format rejects the whole line.
3	'invalid user' variant	Lines for unknown accounts read 'Failed password for invalid user X', while valid-account lines read 'Failed password for X'. A rigid pattern captures the wrong token as the username.

5.2 Before and After

The flawed pattern assumed the classic daemon name, the old timestamp format, and a single line shape:

```
# BEFORE (broken) — matches nothing on modern OpenSSH
^(\w{3}\s+\d+\s[\d:]+)\s+\S+\s+sshd\[\d+\]:\s+
Failed password for (\S+) from (\d+\.\d+\.\d+\.\d+)
```

The corrected pattern makes the daemon suffix optional, captures the ISO 8601 timestamp as a single token (later parsed with datetime.fromisoformat), and treats the "invalid user " prefix as optional:

```
# AFTER (working)
^(?P<timestamp>\S+)\s+\S+\s+sshd(?:-session)?\[\d+\]:\s+
Failed password for (?:invalid user )?(?P<user>\S+)\s+
from (?P<ip>\d{1,3}(?:\.\d{1,3}){3})\s+port\s+\d+
```

The three targeted changes — sshd(?:-session)?, a single-token ISO timestamp, and the optional (?:invalid user)? prefix — allow the parser to match every failed-login line in the log regardless of which format variant it uses. This resolved the detection failure completely.

6. Results

Running the detector against the live authentication log produced the following output:

```
[*] Parsed 70 failed login attempt(s) from /var/log/auth.log
[*] Rule: >= 5 failures within 60s from one IP

[ALERT] Brute-force from 127.0.0.1
        8 failed attempts within 60s (total 70 from this IP)
        window: 2026-06-30 12:58:35.760400-04:00 ->
                2026-06-30 12:58:39.156306-04:00
        targeted user(s): invalid_user, kalibaba, victim
```

The result confirms successful detection. Three points are worth highlighting:

- Correct parsing across formats: all 70 failed attempts were recognised, including both the earlier 'invalid_user' attempts and the recent 'victim' attempts — proof the corrected regex handles every line variant.
- Densest-window selection: the flagged burst (8 failures in roughly 3.4 seconds) is the tightest cluster in the entire log, not merely the most recent activity. The sliding-window logic correctly located the peak across multiple attack sessions.
- Source-based, not account-based: the targeted-user list includes the real account kalibaba alongside the test accounts. The detector flags the behaviour of the source IP regardless of whether the targeted accounts are valid, which is the correct approach for intrusion detection.

7. Active Response & Mitigation

Detection alone is passive: it names an attacker but takes no action. To complete the defensive loop, the detector was extended with an active response mode (--block) that automatically inserts an iptables firewall rule to drop all traffic from any flagged IP. This section documents how a safely-blockable attacker address was generated, the technical obstacles overcome, the blocking logic and its safeguards, and the verification that the block is enforced. A comparison with the production-standard tool fail2ban closes the section.

7.1 Generating a Safely-Blockable Attacker IP

The original attack targeted 127.0.0.1 (loopback). Blocking that address is not an option — it would sever the host's own internal communications. A distinct, disposable source address was therefore required. Rather than provisioning a second virtual machine, a lightweight single-host approach was used: a secondary loopback alias, 127.0.0.2, was bound to the lo interface.

```
sudo ip addr add 127.0.0.2/8 dev lo
ip addr show lo
```

The interface then reported inet 127.0.0.2/8 scope host secondary lo alongside the existing 127.0.0.1/8. Blocking 127.0.0.2 is completely safe: no critical service depends on it, and the primary loopback remains untouched.

7.2 The Source-Binding Challenge

A subtle obstacle emerged. Simply pointing the attack at 127.0.0.2 was not enough — the kernel still selected 127.0.0.1 as the connection's source address, so sshd logged the failures from 127.0.0.1. This was confirmed before wasting an attack run, using the route-selection check:

```
ip route get 127.0.0.2
  -> local 127.0.0.2 dev lo  src 127.0.0.1   (wrong source)

ip route get 127.0.0.2 from 127.0.0.2
  -> local 127.0.0.2 ... (source accepted when forced)
```

The fix was to force the source address explicitly. OpenSSH's -b (bind) flag sets the origin address of the outgoing connection, so ssh -b 127.0.0.2 causes sshd to see — and log — 127.0.0.2 as the source. Because Hydra offers no source-bind option, the failed logins for this phase were generated with a short ssh loop driven by sshpass, which supplies each wrong password non-interactively.

```
# source-bound attack loop (ssh -b sets the logged source IP)
while read pass; do
  sshpass -p "$pass" ssh -n -b 127.0.0.2 -o StrictHostKeyChecking=no \
    -o ConnectTimeout=3 victim@127.0.0.2 exit 2>/dev/null
done < ~/wordlist.txt
```

A single bound test attempt confirmed the fix before the full run: the attempt returned "Permission denied" (proof authentication was reached) and the log recorded the source correctly as 127.0.0.2.

Figure 10. *Bound test attempt returning Permission denied, with auth.log now recording the source as 127.0.0.2.*

7.3 The Blocking Logic and Its Safeguards

The detector was upgraded with a block_ip() function and a --block command-line flag. When blocking is armed, each flagged IP is passed to iptables for a DROP rule. Three safeguards make the automation safe to run:

- Allowlist — a hard-coded set (127.0.0.1, plus any -a additions) that is never blocked, so the tool cannot lock the operator out of the host.
- Idempotency — before adding a rule, iptables -C checks whether one already exists for that IP, preventing duplicate rules on repeated runs.
- Auditing — every ban is printed with a timestamp and appended to /var/log/detector_blocks.log, producing a documented evidence trail.

A further design point: the rule is added with iptables -I (insert at the top of the INPUT chain) rather than -A (append). This guarantees the DROP is evaluated before any pre-existing "accept" rule that could otherwise shadow it. The full upgraded source is in Appendix B.

7.4 Detect-Only Dry Run

Before arming the firewall, the upgraded script was run without --block to confirm it still detects correctly and defaults to passive mode. The output reported Mode: DETECT only and flagged 127.0.0.2 (7 failures in the 09:19 window) with no firewall changes made.

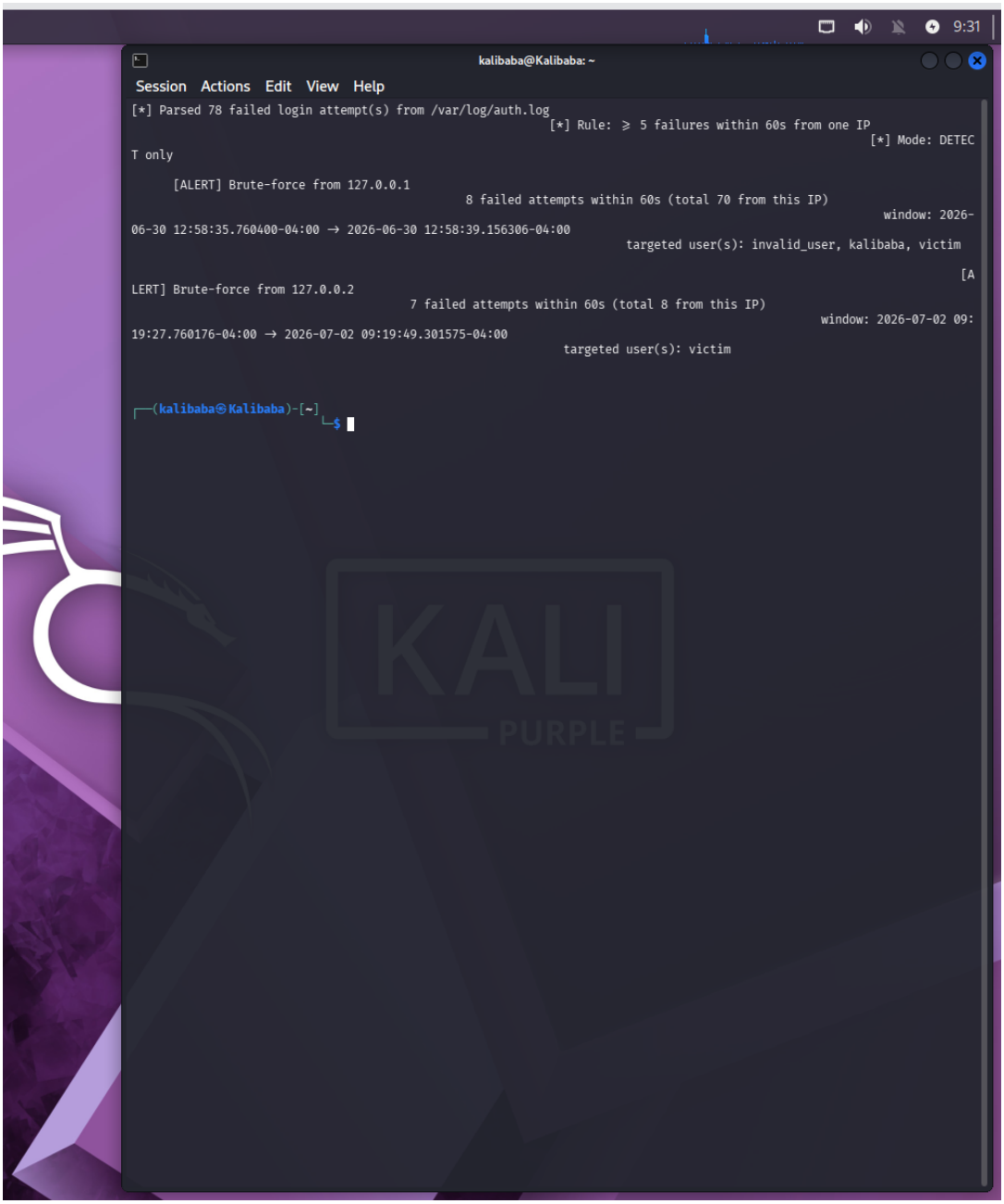

Figure 11. *Detect-only dry run: Mode reads DETECT only and 127.0.0.2 is flagged, with no firewall action taken.*

7.5 Arming the Active Response

The script was then re-run with --block. The mode line changed to BLOCK (active response). The allowlist correctly skipped 127.0.0.1, and 127.0.0.2 was actively banned with a timestamped confirmation.

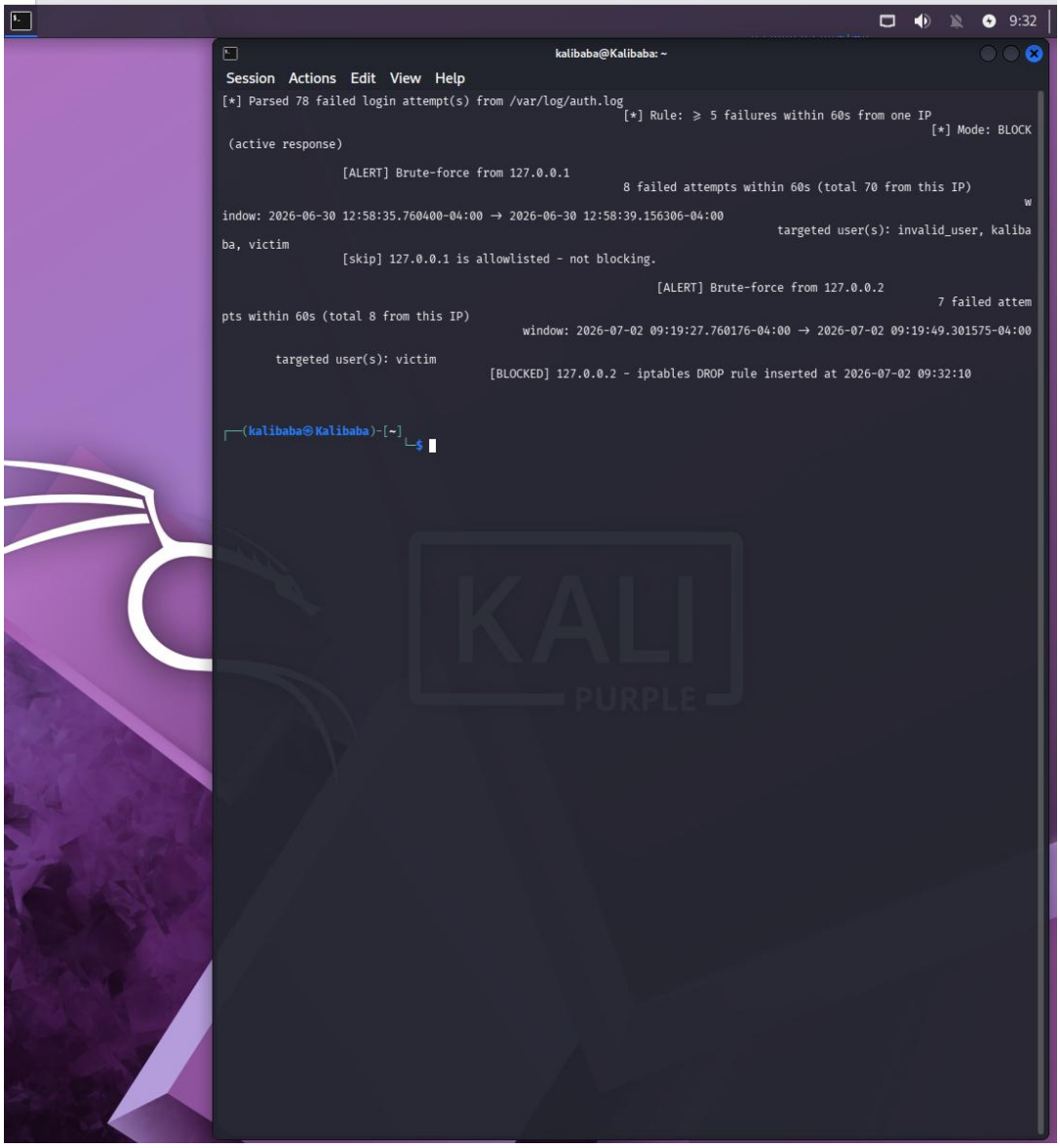

Figure 12. *Active response: 127.0.0.1 skipped by the allowlist; 127.0.0.2 blocked with an iptables DROP rule.*

7.6 Verifying the Block

The ban was confirmed three independent ways. First, the inserted rule was listed at position 1 of the INPUT chain (DROP all -- 127.0.0.2). Second, a connection attempt to 127.0.0.2 — the very command that succeeded minutes earlier — now timed out, proving traffic is being dropped. Third, the audit log contained the ban record with its timestamp.

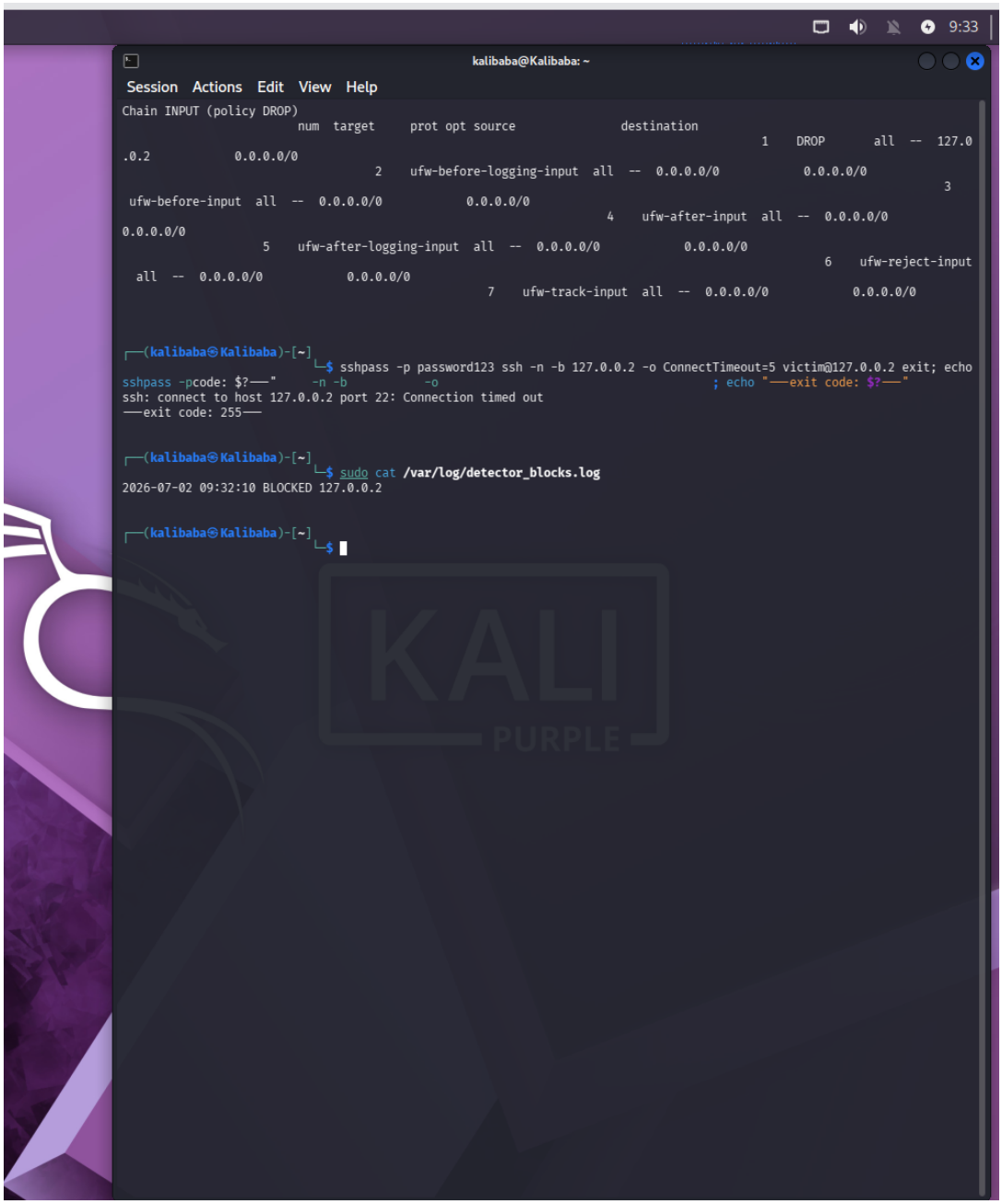

Figure 13. *Three-part proof: the DROP rule at position 1, the now-timed-out connection attempt, and the audit-log entry.*

The before/after contrast is decisive: the identical bound connection succeeded at 09:19 and timed out at 09:33 after the block was applied. The tool now performs a complete detect -> alert -> block -> verify cycle.

7.7 Production Comparison: fail2ban

The custom script demonstrates the response mechanism clearly, but a production environment would typically use fail2ban, a mature daemon that watches logs continuously and manages bans automatically. The two approaches are compared below.

Aspect	This Script (detector.py)	fail2ban
Execution	Run on demand against a static log	Runs continuously as a daemon, tailing logs in real time
Ban logic	Custom sliding-window in Python	Configurable 'jails' with findtime / maxretry / bantime
Ban action	iptables DROP rule	Pluggable actions (iptables, nftables, route, notifications)
Un-ban	Manual (iptables -D)	Automatic ban expiry after bantime
Best use	Learning the mechanism; bespoke logic	Production hardening with minimal custom code

A minimal fail2ban SSH jail expresses the same rule this script implements — ban a source after N failures inside a time window — declaratively:

```
# /etc/fail2ban/jail.local
[sshd]
enabled  = true
port     = ssh
backend  = systemd
maxretry = 5      # like our --threshold
findtime = 60     # like our --window (seconds)
bantime  = 3600   # auto-unban after 1 hour
```

Understanding both is the point: the custom script shows exactly how detection-and-response works under the hood, while fail2ban is what a production system would deploy to do it continuously and safely.

7.8 Reverting the Block

Because the DROP rule is live after the demonstration, it is removed once evidence has been captured, restoring normal traffic to 127.0.0.2:

```
sudo iptables -D INPUT -s 127.0.0.2 -j DROP
```

8. Challenges Encountered

Challenge	Resolution
The initial detector matched zero events despite failed logins being present in the log.	Inspected the raw log lines directly (Step 6) rather than assuming a format. Identified three modern-OpenSSH differences and rewrote the regex accordingly (Section 5).
Process name sshd-session broke patterns anchored on sshd.	Made the daemon suffix optional with sshd(?:-session)?.
ISO 8601 timestamps were incompatible with classic syslog date parsing.	Captured the timestamp as one token and parsed it with datetime.fromisoformat().
'invalid user' lines misaligned the username capture group.	Added an optional (?:invalid user)? prefix to the pattern.

Challenge	Resolution
Detected source is 127.0.0.1 because the attack ran over loopback.	For detection this is an expected lab artifact. For the blocking phase, a separate loopback alias (127.0.0.2) was created so a distinct, non-critical address could be safely banned.
Attacking 127.0.0.2 still logged the source as 127.0.0.1.	Diagnosed with 'ip route get' that the kernel chose 127.0.0.1 as the source. Forced the correct origin with ssh -b 127.0.0.2 (source bind), driven by sshpass since Hydra has no bind option.
The SSH service silently stopped mid-phase (service is 'disabled').	Identified via 'Connection refused' + systemctl status showing inactive; restarted with systemctl start ssh. Noted that systemctl enable ssh would make it persistent.
A default 'accept' rule could shadow a blocking rule.	Used iptables -I (insert at top of INPUT) instead of -A (append), so the DROP rule is evaluated first. Verified it landed at position 1.
Automated blocking risks locking the operator out.	Built in an allowlist (never blocks 127.0.0.1 or -a additions), an idempotency check to avoid duplicate rules, and a timestamped audit log for every ban.

9. Conclusion

This project delivered a complete SSH brute-force detection-and-response capability. A controlled attack was generated with Hydra, the resulting log evidence was verified, and a Python detector was built and confirmed to flag the attack against the live authentication log. The tool was then extended from passive detection into active mitigation: an iptables blocking mode that automatically bans an offending source, guarded by an allowlist, an idempotency check, and an audit trail. A source-bound loopback alias made this demonstrable safely on a single host, and the block was proven enforced three independent ways. The key learning outcome was the value of validating assumptions against real data — first when the detector only worked after the true log format was inspected, and again when the block only worked after diagnosing the kernel's source-address selection. The result is a tool that performs a full detect, alert, block, and verify cycle, with fail2ban documented as the production-grade equivalent.

Appendix A — Defense Questions & Answers

Q1. Why did the original regex fail to detect anything?

A. It assumed the classic 'sshd[PID]' process name, the old 'Mon DD HH:MM:SS' syslog timestamp, and a single line shape. Modern OpenSSH (9.8+) logs a per-connection worker called 'sshd-session', uses ISO 8601 timestamps, and has a separate 'invalid user' line variant — so the pattern matched none of the real lines.

Q2. What is the difference between sshd and sshd-session?

A. As of OpenSSH 9.8 the monolithic daemon was split for security. The listener process 'sshd' accepts connections and hands each one to an unprivileged per-connection worker, 'sshd-session', which handles authentication. That worker is what writes the 'Failed password' entries on modern systems.

Q3. How does the sliding-window detection work?

A. Events for each IP are sorted by time. Two pointers scan the list; the end pointer advances one event at a time, and the start pointer moves up whenever the time between start and end exceeds the window. The maximum number of events between the pointers at any moment is the densest burst for that IP. If it meets the threshold, an alert fires.

Q4. Why is the threshold 5 within 60 seconds?

A. It is a balance between sensitivity and false positives. A legitimate user rarely fails five logins in one minute, whereas an automated attack easily does. Both values are command-line configurable (-t and -w), so the rule can be tuned per environment.

Q5. Why is the detected IP 127.0.0.1?

A. The attack was launched against the local loopback interface, so the source recorded by SSH is 127.0.0.1. In a real deployment the source would be the attacker's remote IP; the detection logic is identical and would group and flag that address the same way.

Q6. Why does the alert include the real account 'kalibaba'?

A. The detector flags the behaviour of a source IP, not of a specific account. An attacker often sprays many usernames, valid and invalid. Grouping by source IP catches the attack regardless of which accounts were targeted, which is why kalibaba appears alongside the test accounts.

Q7. How would you extend this detector for production use?

A. Run it continuously (tail the log or read the journal in real time), integrate with a blocklist tool such as fail2ban or an iptables rule to respond automatically, forward alerts to a SIEM, and add allowlisting for trusted hosts to suppress known-good sources.

Q8. Could an attacker evade this detector?

A. Yes — by slowing attempts below the threshold ('low and slow'), by spreading attempts across many source IPs, or by using valid-looking traffic. Mitigations include longer observation windows, aggregating across subnets, and correlating with other signals such as successful logins after many failures.

Q9. Why create a 127.0.0.2 loopback alias instead of just blocking 127.0.0.1?

A. Blocking 127.0.0.1 would cut off the host's own internal communications, since countless local services rely on it. The alias 127.0.0.2 gives a distinct, disposable source address that can be safely dropped without affecting anything critical, all on a single machine.

Q10. Why was ssh -b needed, and why not just use Hydra for the blocking phase?

A. Pointing the attack at 127.0.0.2 wasn't enough — the kernel still chose 127.0.0.1 as the connection's source, so sshd logged the wrong IP. The -b flag binds the source address so 127.0.0.2 is the logged origin. Hydra has no source-bind option, so a short ssh loop with sshpass was used to generate the failed logins from the bound address.

Q11. What stops the blocking script from locking you out of your own machine?

A. An allowlist that is checked before any ban: 127.0.0.1 (and any address added with -a) is never blocked. The run output shows 127.0.0.1 being explicitly skipped. There is also an idempotency check so repeated runs don't stack duplicate rules.

Q12. Why insert the iptables rule with -I instead of -A?

A. -I inserts at the top of the INPUT chain, so the DROP is evaluated before any earlier 'accept' rule that might otherwise match the traffic first and shadow the block. Listing the chain confirmed the rule landed at position 1.

Q13. How is this different from fail2ban, and when would you use each?

A. fail2ban is a production daemon that tails logs continuously and manages bans automatically, with configurable jails and automatic un-banning after a set time. The custom script runs on demand and demonstrates the detect-and-respond mechanism explicitly. The script is ideal for learning and bespoke logic; fail2ban is what you'd deploy to harden a real server.

Q14. How would you prove the block actually worked?

A. Three ways, all captured: list the firewall rule (iptables -L shows DROP for 127.0.0.2 at position 1); attempt the same connection that previously succeeded and observe it now time out; and check the audit log (/var/log/detector_blocks.log) for the timestamped ban entry.

Appendix B — Full Source: detector.py

```
#!/usr/bin/env python3
"""
Brute Force Detection & Response - CEH Final Project 4
Parses SSH auth logs, counts failed logins per source IP inside a sliding
time window, flags any IP over a threshold, and (optionally) blocks it with
an iptables DROP rule.
"""

import re
import sys
import argparse
import subprocess
from datetime import datetime, timedelta
from collections import defaultdict

# Handles: classic 'sshd' AND modern 'sshd-session';
# valid-user AND 'invalid user' lines; ISO 8601 timestamp.
LINE_RE = re.compile(
    r'^(?P<timestamp>\S+)\s+\S+\s+sshd(?:-session)?\[\d+\]:\s+'
    r'Failed password for (?:invalid user )?(?P<user>\S+)\s+'
    r'from (?P<ip>\d{1,3}(?:\.\d{1,3}){3})\s+port\s+\d+'
)

# IPs the tool must NEVER block, to avoid locking the operator out.
ALLOWLIST = {"127.0.0.1"}


def parse_log(path):
    """Return a list of (datetime, ip, user) for every failed login."""
    events = []
    with open(path, "r", errors="replace") as fh:
        for line in fh:
            m = LINE_RE.search(line)
            if not m:
                continue
            try:
                ts = datetime.fromisoformat(m.group("timestamp"))
            except ValueError:
                continue
            events.append((ts, m.group("ip"), m.group("user")))
    return events


def detect(events, threshold, window):
    """Flag an IP if any `window`-second span holds >= threshold failures."""
    by_ip = defaultdict(list)
    for ts, ip, user in events:
        by_ip[ip].append((ts, user))

    alerts = []
```

```
    win = timedelta(seconds=window)
    for ip, attempts in by_ip.items():
        attempts.sort(key=lambda x: x[0])
        times = [a[0] for a in attempts]
        start = 0
        best = 0
        best_span = None
        for end in range(len(times)):
            while times[end] - times[start] > win:
                start += 1
            count = end - start + 1
            if count > best:
                best = count
                best_span = (times[start], times[end])
        if best >= threshold:
            users = sorted({u for _, u in attempts})
            alerts.append({
                "ip": ip,
                "count": best,
                "total": len(attempts),
                "span": best_span,
                "users": users,
            })
    return alerts


def is_blocked(ip):
    """Return True if an iptables DROP rule for this IP already exists."""
    r = subprocess.run(
        ["iptables", "-C", "INPUT", "-s", ip, "-j", "DROP"],
        capture_output=True,
    )
    return r.returncode == 0


def block_ip(ip, allow):
    """Insert an iptables DROP rule for ip, honouring the allowlist."""
    if ip in allow:
        print(f"        [skip] {ip} is allowlisted - not blocking.")
        return False
    if is_blocked(ip):
        print(f"        [skip] {ip} already blocked - no duplicate added.")
        return False
    # -I inserts at the TOP of INPUT so a default 'accept lo' rule can't shadow it.
    r = subprocess.run(
        ["iptables", "-I", "INPUT", "-s", ip, "-j", "DROP"],
        capture_output=True, text=True,
    )
    if r.returncode == 0:
        stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        print(f"        [BLOCKED] {ip} - iptables DROP rule inserted at {stamp}")
        with open("/var/log/detector_blocks.log", "a") as fh:
            fh.write(f"{stamp} BLOCKED {ip}\n")
        return True
```

```
    print(f"        [error] failed to block {ip}: {r.stderr.strip()}")
    return False


def main():
    ap = argparse.ArgumentParser(description="SSH brute-force detector & responder")
    ap.add_argument("-f", "--file", default="/var/log/auth.log",
                    help="auth log to parse (default: /var/log/auth.log)")
    ap.add_argument("-t", "--threshold", type=int, default=5,
                    help="failures within the window to alert (default: 5)")
    ap.add_argument("-w", "--window", type=int, default=60,
                    help="time window in seconds (default: 60)")
    ap.add_argument("--block", action="store_true",
                    help="actively block flagged IPs with iptables (needs sudo)")
    ap.add_argument("--allow", action="append", default=[],
                    help="extra IP to never block (repeatable)")
    args = ap.parse_args()

    allow = set(ALLOWLIST) | set(args.allow)

    try:
        events = parse_log(args.file)
    except FileNotFoundError:
        print(f"[!] Log file not found: {args.file}")
        sys.exit(1)
    except PermissionError:
        print(f"[!] Permission denied reading {args.file} - run with sudo")
        sys.exit(1)

    print(f"[*] Parsed {len(events)} failed login attempt(s) from {args.file}")
    print(f"[*] Rule: >= {args.threshold} failures within {args.window}s from one IP")
    print(f"[*] Mode: {'BLOCK (active response)' if args.block else 'DETECT only'}\n")

    alerts = detect(events, args.threshold, args.window)
    if not alerts:
        print("[+] No brute-force activity detected.")
        return

    alerts.sort(key=lambda a: a["count"], reverse=True)
    for a in alerts:
        s, e = a["span"]
        print(f"[ALERT] Brute-force from {a['ip']}")
        print(f"        {a['count']} failed attempts within {args.window}s "
              f"(total {a['total']} from this IP)")
        print(f"        window: {s} -> {e}")
        print(f"        targeted user(s): {', '.join(a['users'])}")
        if args.block:
            block_ip(a["ip"], allow)
        print()


if __name__ == "__main__":
    main()
```

Appendix C — Reproduction & Verification Playbook

This playbook lets an evaluator reproduce the entire detection-and-mitigation cycle under identical lab conditions on the Kali host. Run the phases in order.

Phase 1 — Reset the Environment State

Clear any leftover demonstration state so verification starts clean: remove a prior block if present, and make sure the SSH server is running.

```
# Remove any existing demo block (ignore error if none exists)
sudo iptables -D INPUT -s 127.0.0.2 -j DROP 2>/dev/null

# Ensure the OpenSSH server is running and listening
sudo systemctl start ssh
```

Phase 2 — Simulate the Attack

Generate the failed-login evidence, source-bound to 127.0.0.2 so sshd records the correct attacker address.

```
while read -r pass; do
  sshpass -p "$pass" ssh -n -b 127.0.0.2 -o StrictHostKeyChecking=no \
    -o ConnectTimeout=3 victim@127.0.0.2 exit 2>/dev/null
done < ~/wordlist.txt
```

Phase 3 — Passive Detection (Dry Run)

Confirm the parser processes the modern log lines and flags the attacker without touching the firewall.

```
sudo python3 ~/detector.py -t 5 -w 60
```

Expected: the header shows Mode: DETECT only, and a brute-force alert is raised for 127.0.0.2 with the dense-window details and targeted user.

Phase 4 — Arm the Active Response

Re-run with blocking armed.

```
sudo python3 ~/detector.py -t 5 -w 60 --block
```

Expected: the header shows Mode: BLOCK (active response); 127.0.0.1 is skipped via the allowlist; and a line reads [BLOCKED] 127.0.0.2 - iptables DROP rule inserted at <timestamp>.

Phase 5 — Verify Enforcement (three checks)

1. Rule placement and packet counters — confirm the DROP rule sits at the top of the INPUT chain:

```
sudo iptables -L INPUT -n -v --line-numbers
```

Criteria: rule 1 is a DROP from 127.0.0.2. Note the pkts / bytes counters on the left — after the next step they will have incremented, proving the kernel is actively dropping traffic.

2. Network isolation — attempt the connection that previously succeeded:

```
ssh -b 127.0.0.2 -o ConnectTimeout=5 victim@127.0.0.2
```

Criteria: the connection fails to establish and times out within about 5 seconds with "ssh: connect to host 127.0.0.2 port 22: Connection timed out", rather than prompting for a password. Re-running the iptables command above will now show higher pkts/bytes counters.

3. Audit trail — confirm the ban was recorded (the log is root-owned, so use sudo):

```
sudo cat /var/log/detector_blocks.log
```

Criteria: the file contains a timestamped line reading <timestamp> BLOCKED 127.0.0.2.

Phase 6 — Clean Up

Remove the demonstration block to restore normal traffic to 127.0.0.2:

```
sudo iptables -D INPUT -s 127.0.0.2 -j DROP
```